Reminders and announcements

- During lectures you can submit questions to: vevox.app ID: 107-018-952
- You can find answers to questions in the COMP1870 Teams Channel (link in Minerva) (along with other discussion).
- Maths Lunch Hours, Mondays and Wednesdays, 1.00pm - 2.00pm. Bragg GR.18.
- My student hour: Tuesdays 1.00pm - 2.00pm, Bragg building foyer.

Reminders and announcements

Starting on Thursday 4 November, COMP1870 lectures change to:

- Thursdays from 1.00pm to 3.00pm in
  Chem. & Process Engineering Lecture Theatre A (2.23)
  (Starting from Thursday 4 November)
- Mondays from 4.00pm to 6.00pm in
  Mechanical Engineering Lecture Theatre A (1.33)
  (Starting from Monday 9 November)

Revision

- If p and q are integers, then $p|q$ means that $q = kp$ for some $k \in \mathbb{Z}$. We say that p **divides** q, or p is a **factor** of q, or q is a **multiple** of p.
- (It is natural to say that p divides q if $\frac{q}{p}$ is an integer, but this definition creates problems with division by zero.)
- If p is any integer, then $p|0$, $1|p$, and $p|p$.
- If $p \neq 0$, then $0 \nmid p$.
- If $p|a$, then $p|(ab)$, for any integer b.
- If $p|a$ and $p|b$, then $p|(a+b)$.

Properties of divisibility

Proposition 9.3. Let A be a set of positive integers. Consider the following relation R on the elements of A. If a and b are positive integers then we declare (a, b) to be in R if and only if a divides b. Then R is a partial order.

Proof. We must prove R is reflexive, anti-symmetric, and transitive.

Reflexive: We must prove, for every $a \in A$, that $(a,a) \in R$. We let a be an arbitrary member of A. Because $a|a$, (a,a) is in R.

Anti-symmetry: Assume (a,b), $(b,a) \in R$. Then $b = ma$ and $a = nb$ for some $m, n \in \mathbb{Z}$. Thus $b = ma = m(nb) = (mn)b$. Now $mn = 1$, implying $m = n = 1$ or $m = n = -1$.

Properties of divisibility

Proposition 9.3. Let A be a set of positive integers. Consider the following relation R on the elements of A. If a and b are positive integers then we declare (a, b) to be in R if and only if a divides b. Then R is a partial order.

We can eliminate $m=n=-1$, since $b=ma$ and a,b are in A, so are positive. Therefore $m=n=1$, so $b=ma=a$.

Transitive: Assume $(a,b)\in R$ and $(b,c)\in R$. Therefore $b=ma$ and $c=nb$ for some $m,n\in\mathbb{Z}$. Thus $c=nb=n(ma)=(nm)a$. Since $nm\in\mathbb{Z}$, this implies $a|c$, so $(a,c)\in R$

□

Number representations

When we represent an integer with the expression 41025, we mean the integer

$$4 \cdot 10^4 + 1 \cdot 10^3 + 0 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0.$$

This is **base-10** (**decimal**) representation.

We can represent integers using other **bases**. If $b > 1$ is an integer, then the expression $(a_n a_{n-1} \cdots a_1 a_0)_b$ represents

$$a_n b^n + a_{n-1} b^{n-1} + \cdots + a_1 b^1 + a_0 b^0.$$

where each a_i is a **digit** between 0 and $b - 1$.

Number representations

Common choices of bases are:

- 2 (**binary** representation),
- 8 (**octal** representation),
- 10 (**decimal** representation),
- 16 (**hexadecimal** representation).

In hexadecimal representation, we need digits for all of the numbers $0, 1, 2, \ldots, 15$.

A	B	C	D	E	F
10	11	12	13	14	15

$(B4F)_{16} = 11 \cdot 16^2 + 4 \cdot 16^1 + 15 \cdot 16^0$

Decimal representations are often written without brackets.

Number representations

In the representation $(a_n a_{n-1} \cdots a_1 a_0)_b$, we say a_n is the **leading digit**. We always assume the leading digit is non-zero. For example, we would write $(401)_{10}$ rather than $(0401)_{10}$.

Proposition 9.4. Let b be an integer greater than 1 and let m be a non-negative integer. Then m has exactly one base-b representation.

Proof. We must prove a representation exists and then prove it is unique. We leave the first part as an exercise, and prove uniqueness. Assume for a contradiction that m has two different representations. If they have different numbers of digits, we pad the representation with a smaller number with zeroes.

Number representations

Therefore we assume $(a_s a_{s-1} \cdots a_0)_b$ and $(a'_s a'_{s-1} \cdots a'_0)_b$ are different representations of n. Because they are different, there must be an index i such that $a_i \neq a'_i$. Let i be the smallest index such that $a_i \neq a'_i$.

$$0 = n - n = (a_s b^s + a_{s-1} b^{s-1} + \cdots + a_0 b^0) - (a'_s b^s + a'_{s-1} b^{s-1} + \cdots + a'_0 b^0)$$

$$= (a_s - a'_s) b^s + (a_{s-1} - a'_{s-1}) b^{s-1} + \cdots + (a_0 - a'_0) b^0$$

$$= (a_s - a'_s) b^s + (a_{s-1} - a'_{s-1}) b^{s-1} + \cdots + (a_i - a'_i) b^i$$

and $(a_{i-1} - a'_{i-1}) = 0$, and similarly with all smaller terms.

Number representations

We subtract $(a_i - a_i')b^i$ from both sides.

$$(a_i' - a_i)b^i = (a_s - a_s')b^s + \cdots + (a_{i+1} - a_{i+1}')b^{i+1}$$

Divide both sides by b^i.

$$a_i' - a_i = (a_s - a_s')b^{s-i} + \cdots + (a_{i+1} - a_{i+1}')b$$

b is a factor of the right hand side, so $b \mid (a_i' - a_i)$. Since a_i', a_i are in $\{0, 1, \ldots, b-1\}$, $a_i' - a_i$ is in $-(b-1), \ldots, -1, 0, 1, \ldots, b-1$. The only multiple of b in this range is 0, so $a_i' - a_i = 0$, contradicting $a_i \neq a_i'$. $\square$

Logarithms

Proposition 9.6. Let b be an integer greater than 1. If m is a non-negative integer, and n is the smallest integer such that $b^n > m$, then it takes exactly n digits to represent m in base b.

$99 = 64 + 32 + 2 + 1$ so

$(99)_{10} = (1100011)_2$.

Also, $128 = 2^7$ is the smallest power of two greater than 99, so 99 needs 7 base-2 digits.

Logarithms

Recall that a **logarithm** is an inverse of an exponential function.

$$\log_b x = y \text{ means } b^y = x.$$

Thus $\log_b x$ is the number we must raise b to in order to obtain x. We say that b is the **base** of the logarithmic function $\log_b$.

$\log_{10} 1000 = 3$ $\log_2 256 = 8$ $\log_7 7 = 1$

Proposition 9.8. Let b be an integer greater than 1. If m is a positive integer, then the number of digits in the base-b representation of m is $\lfloor \log_b m \rfloor + 1$.

Logarithms

We can convert between logarithms with different bases by multiplying by a constant:

Exercise. Let a and b be integers greater than 1. Then

$$\log_b m = \frac{1}{\log_a b}(\log_a m).$$

$$\log_2 100 = \frac{1}{\log_{10} 2} \log_{10} 100 = \frac{1}{0.3010} 2$$

$$= 6.644\ldots$$

All logarithms are the same, up to multiplying by a constant. For this reason, we often say that the number of digits needed to represent m is $\log m$, without specifying the base.

Polynomial-time algorithms

We want algorithms with running times that look like this:

$$(\text{input size})^2 \text{ or } (\text{input size})^3 \text{ or, generally } (\text{input size})^c$$

(where c is a constant number).

These are called **polynomial-time algorithms**.

We do not want algorithms with running times that look like this:

$$2^{(\text{input size})} \text{ or } (\text{input size})!$$

Polynomial-time algorithms for arithmetic

Question. What is the input size of the integer m?

It is not m, since we don't need m digits to represent m.

Important point. The input size of the integer m is $\log m$.

We need algorithms which will carry out arithmetic operations with a running time of the form $(\log m)^c$.

If an algorithm is polynomial-time with respect to representation in one base, it is polynomial-time with respect to representations in any base.

Adding integers

The addition and multiplication algorithms we learn in school are polynomial-time algorithms.

Example. Add $(6034)_8$ and $(127)_8$.

$$\begin{array}{r} (6\ 0\ 3\ 4)_8 \\ +\quad (1\ 2\ 7)_8 \\ \hline (6\ 1\ 1\ 6\ 3)_8 \end{array}$$

Multiplying integers

Example. Multiply $(704)_8$ by $(25)_8$.

$$\begin{array}{r} (7\ 0\ 4)_8 \\ \times \qquad (5)_8 \\ \hline (4\ 3^{2}\ 2\ 4)_8 \end{array}$$

$$\begin{array}{r} (7\ 0\ 4)_8 \\ \times \qquad (2)_8 \\ \hline (1\ 6\ {}^{1}1\ 0)_8 \end{array}$$

$$\begin{array}{r} (1\ 6\ 1\ 0\ 0)_8 \\ + \quad (4\ 3\ 2\ 4)_8 \\ \hline ({}^{1}2\ 2\ 4\ 2\ 4)_8 \end{array}$$

Exponentiation

Question How do we calculate something like $(11)_2^{(1010)_2}$?

Note that we cannot multiply together $(1010)_2 = (10)_{10}$ copies of $(11)_2 = (3)_{10}$, because this does not lead to a polynomial-time algorithm!

Remember that the input size of $(1010)_2$ is not 10. It is 4, because the number can be represented with 4 binary digits.